BCSE308P – COMPUTER NETWORKS LAB

PROTOCOL LAB: AN INTERACTIVE COMPUTER NETWORKS SIMULATOR

A Project Report

submitted in partial fulfilment of the requirements for the course

Submitted by

SAVIR GHAI
24BCT0309

Team: F1_Team34

Faculty: Prof. SREETHAR S

B.Tech. in Computer Science and Engineering

School of Computer Science and Engineering

October 2026

ABSTRACT

Computer networking courses teach protocols through static diagrams and worked numerical examples, yet the behaviour that matters (a frame lost in a sliding window, a count-to-infinity loop, a voice call stuck behind a bulk download) only becomes clear when the conditions can be changed and the outcome observed. Cisco Packet Tracer was not usable for this project, and heavyweight simulators such as ns-3 need installation, scripting and a long setup before a first result.

This report presents **Protocol Lab**, a zero-install browser simulator written in plain HTML, CSS and JavaScript. It contains eight interactive labs that follow the course syllabus from the layered architecture to queuing and quality of service: packet journey, error detection and correction, flow control, multiple access, subnetting, routing, TCP congestion control, and queuing with QoS. Every lab keeps its calculations in pure functions separate from the screen code, so the same logic runs in the browser and in Node.js. A fixed-seed random generator makes every run repeatable.

The contribution is an integrated, verified and reproducible tool that places the baseline and the improved mechanism of the same problem side by side on identical conditions. Four comparisons are evaluated over 30 seeds per scenario: plain distance vector against split horizon and poison reverse, Go-Back-N against Selective Repeat, TCP Tahoe against Reno, and FIFO against priority queuing and weighted fair queuing. At 20% frame loss Selective Repeat raises link efficiency from 0.405 ± 0.047 to 0.681 ± 0.033. At 95% data load, weighted fair queuing cuts the 95th percentile voice delay from 374.3 ± 10.5 ms to 16.2 ± 0.3 ms and removes voice loss. A suite of 78 automated tests checks the labs against known answers such as the CRC-32 check value 0xCBF43926.

Keywords: computer networks, protocol simulation, ARQ, TCP congestion control, queuing, quality of service, teaching tool.

CERTIFICATE

This is to certify that the project report entitled **"Protocol Lab: An Interactive Computer Networks Simulator"** is a bona fide record of the work carried out by **Savir Ghai (24BCT0309)** of Team F1_Team34, B.Tech. in Computer Science and Engineering, School of Computer Science and Engineering, for the course **BCSE308P – Computer Networks Lab**, during the academic term ending October 2026.

The work reported here has not been submitted, in part or in full, for any other course or degree.

Faculty signature: _____ Student signature: _____

Prof. SREETHAR S Savir Ghai, 24BCT0309

Date: ____ / ____ / 2026

TABLE OF CONTENTS

Sl.No	Contents	Page No.
1	**INTRODUCTION**	**1**
1.1	Background and motivation	1
1.2	Problem statement	1
1.3	Objectives	1
1.4	Scope and syllabus coverage	1
1.5	Organisation of the report	2
2	**LITERATURE SURVEY**	**3**
2.1	Existing approaches for learning and simulating protocols	3
2.2	Related work	3
2.3	Research gap and novelty	3
3	**PROPOSED SYSTEM**	**4**
3.1	System architecture	4
3.2	Baseline mechanisms	4
3.3	Proposed mechanisms and algorithms	4
3.4	Novelty statement	5
3.5	Assumptions and design decisions	5
4	**IMPLEMENTATION**	**7**
4.1	Tools, platform and versions	7
4.2	Simulation setup and parameters	7
4.3	Code structure	7
4.4	Testing and verification	8
4.5	Challenges and how they were solved	9
5	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	**10**
5.1	Experimental setup	10
5.2	Scenarios and parameters	10
5.3	Results	10
5.4	Analysis	13
5.5	Critical evaluation	13
6	**CONCLUSION AND FUTURE WORK**	**15**
6.1	Summary of results	15
6.2	Limitations	15
6.3	Future work	15
7	**REFERENCES**	**16**
A	**Appendix A: Code and configuration listings**	**17**
B	Appendix B: Screenshots, raw results and test output	20

1 Introduction

1.1 Background and motivation

A first course in computer networks covers a long list of mechanisms: the layered architecture, error detection and correction, sliding-window flow control, medium access, addressing and subnetting, routing, congestion control and quality of service. Textbooks explain each one with a diagram and one worked example. What the diagrams cannot show is how the mechanism behaves when the conditions change: what happens to Go-Back-N when the loss rate rises from 2% to 20%, why a distance vector network takes 31 rounds to notice that a router has gone away, or why a voice call becomes unusable once a bulk transfer shares its queue.

Professional simulators answer such questions, but they also bring a cost. ns-3 and Mininet need a Linux installation, scripting in C++ or Python and some time before the first result appears; Cisco Packet Tracer was not usable for this project. A student preparing for a lab examination or a viva needs something lighter: open one file, move a slider, watch the protocol react, and trust the answer.

1.2 Problem statement

To design and build a self-contained, zero-install simulator that covers the main mechanisms of the Computer Networks syllabus, lets the user change the conditions of each experiment, and presents each mechanism next to its simpler baseline on identical input, so that the benefit of the improvement can be measured and not only described. The results must be correct (checked against known answers) and repeatable (the same seed always gives the same numbers).

1.3 Objectives

1. Build eight interactive labs that map onto Units 1 to 6 of the syllabus, in plain HTML, CSS and JavaScript with no framework, no build step and no server.
2. Keep every calculation in pure functions, separate from the screen code, so the same logic can be tested in Node.js.
3. Verify the labs against textbook results and invariants with an automated test suite.
4. Evaluate four baseline-versus-improvement pairs (distance vector, ARQ, TCP, queuing) over repeated seeded runs and report mean ± standard deviation.
5. Document the models and their limits honestly, so that the simulator is not mistaken for a production-grade network simulator.

1.4 Scope and syllabus coverage

Table 1.1 shows which unit each lab serves. Unit 7 (DNS, HTTP and FTP) is the only unit without a lab and is listed as future work.

Table 1.1: Labs and the syllabus units they cover

Lab	Unit	Topics covered
Packet journey	1, 2	Encapsulation through TCP, IPv4 and Ethernet across two routers; TTL, MAC rewriting, checksums, CRC-32; circuit against packet switching delay; statistical multiplexing
Error detection	3	Hamming code (single-error correcting and SEC-DED), CRC with step-by-step division, Internet checksum, Monte Carlo detection test
Flow control	3	Stop-and-Wait, Go-Back-N and Selective Repeat on a lossy link; space-time diagram; efficiency sweeps

Lab	Unit	Topics covered
Multiple access	3	Pure and slotted ALOHA, 1-persistent and non-persistent CSMA, CSMA/CD; throughput against offered load
Subnetting	4	Address calculator, equal and VLSM subnet planning, same-subnet test, route summarisation
Routing	5	Dijkstra step by step; distance vector round by round; broken links; count to infinity; split horizon; poison reverse
TCP congestion	6	Tahoe and Reno window lab; shared bottleneck with drop-tail or RED; fairness
Queuing and QoS	6	FIFO, priority queuing and weighted fair queuing; delay, jitter, loss and throughput of a voice call

1.5 Organisation of the report

Chapter 2 reviews existing tools and related work and states the gap. Chapter 3 describes the architecture, the baseline mechanisms and the improved mechanisms. Chapter 4 covers the implementation, the parameters and the testing. Chapter 5 reports the experiments with mean ± standard deviation over 30 seeds and critically evaluates them. Chapter 6 concludes and lists limitations and future work. The appendices give code listings, screenshots, raw results and test output.

2 Literature Survey

2.1 Existing approaches for learning and simulating protocols

Three families of tools are used to teach and study protocols. **Textbooks and slides** [1]–[3] give the most reliable explanations, but the examples are fixed and cannot be re-run with other parameters. **Emulators and training tools** such as Cisco Packet Tracer let a student build a topology and configure devices; they teach configuration well but hide the internal state of a protocol such as the window of a sender. **Research simulators** such as ns-3 [4] and Mininet [5] model protocols in great detail and are the standard tools for evaluation, but they need installation, scripting and some expertise.

The mechanisms compared in this project are classical and well documented. Selective Repeat and Go-Back-N are textbook ARQ schemes [1], [2]. TCP Tahoe comes from Jacobson's congestion control work [6], fast recovery in Reno from [7], and the comparison of Tahoe, Reno and SACK in simulation from [8]. Fair queuing and its packet approximation come from [9], and Random Early Detection from [10]. Distance vector routing, count to infinity and split horizon are described in RIP [11]. Error detection and correction go back to Hamming [12] and to cyclic redundancy codes [13].

2.2 Related work

Table 2.1: Related work and tools

S. No	Authors / year	Technique	Metrics and tool	Merits	Gaps
1	Henderson, Lacage, Riley, 2008 [4]	ns-3 discrete-event network simulator	Packet-level simulation of full protocol stacks in C++	Accurate, extensible, standard in research	Needs installation and programming; slow to try one idea in class
2	Lantz, Heller, McKeown, 2010 [5]	Mininet network emulation	Runs real network software on virtual hosts and switches	Real code, fast set-up for SDN experiments	Linux and command line needed; not a teaching view of protocol state
3	Fall and Floyd, 1996 [8]	Simulation of Tahoe, Reno and SACK TCP	Throughput and recovery behaviour in ns	Clear evidence for Reno and SACK benefits	Research paper; not interactive
4	Demers, Keshav, Shenker, 1989 [9]	Fair queuing	Delay and throughput per flow in simulation	Shows why FIFO is unfair to small flows	Theory and simulation only
5	Floyd and Jacobson, 1993 [10]	Random Early Detection	Average queue, drop probability, throughput	Avoids synchronised drops and full queues	Parameter tuning is hard; no student-facing tool
6	Cisco Networking Academy (Packet Tracer)	Topology and configuration simulator	Device configuration and connectivity tests	Very good for CLI and topology practice	Not usable in this project; protocol internals hidden

2.3 Research gap and novelty

The mechanisms in this project are not new. The gap is practical: no single lightweight tool lets a student (i) open one file with no installation, (ii) change the conditions of an experiment on the screen, (iii) see the baseline and the improved mechanism side by side on identical input, and (iv) trust the result because it is tested against known answers and repeatable from a seed. Protocol Lab fills that gap for the syllabus of this course. It does not compete with ns-3 on accuracy; its models are simplified on purpose (Section 3.5) and the report says where.

3 Proposed System

3.1 System architecture

Protocol Lab is one HTML page that loads a small core, a canvas chart library and one module per lab (Figure 3.1). Each lab file registers itself with the core through a tab registry, and a hash router shows the selected lab. The calculations live in pure functions at the top of each file, and the screen code, which starts after a guard that exits when no `document` exists, calls those functions. In Node.js the same file loads without the screen part, which is how the tests and the experiments run.

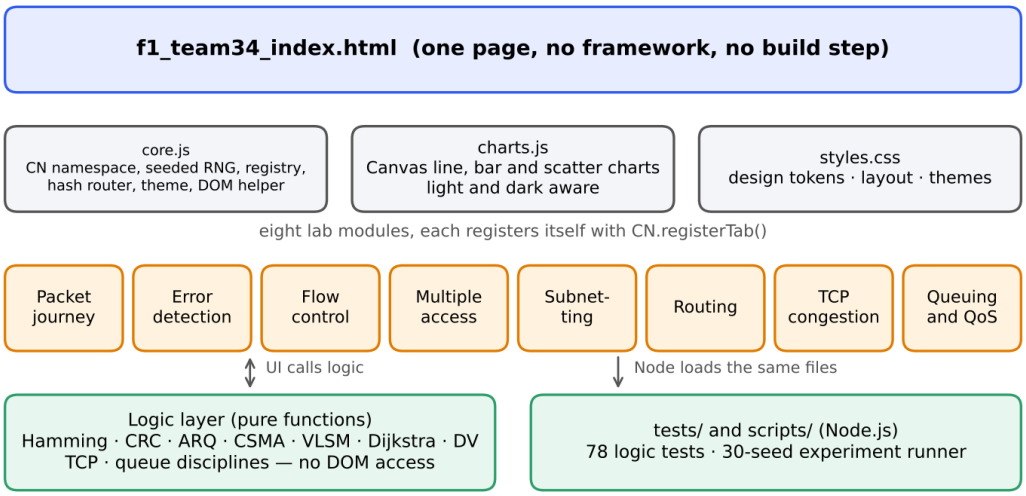


Figure 3.1: Architecture of Protocol Lab

3.2 Baseline mechanisms

For each lab that has an improvement, the baseline is the simpler mechanism from the textbook:

- **Routing:** plain distance vector, where each router advertises its full table to every neighbour. After a link fails, routers keep advertising routes that go through the failed link to each other.
- **Flow control:** Go-Back-N, where the receiver accepts only the next expected frame and the sender repeats the whole window after a loss.
- **TCP congestion control:** Tahoe, which sets the congestion window to 1 after any loss signal, including three duplicate ACKs.
- **Queuing:** FIFO, where packets leave in arrival order, so a small voice packet waits behind every large data packet ahead of it.

3.3 Proposed mechanisms and algorithms

The “proposed” side of each pair is the standard improvement taught in the course. The simulator’s contribution is to run both on the same input and show the difference.

3.3.1 Split horizon and poison reverse

When router n builds the advertisement for neighbour m, it leaves out any destination whose next hop is m (split horizon) or advertises it with an infinite cost (poison reverse). The simulator uses infinity = 32 because the example costs reach 20.

```
for each router n, for each destination d:
    best = INFINITY
    for each neighbour m of n, with link cost c:
        adv = cost[m][d]
        if next_hop[m][d] == n and mode in (split, poison):
            adv = INFINITY            # m learned d through n: do not trust it
        best = min(best, min(INFINITY, c + adv))
    new_cost[n][d] = best
```

3.3.2 Selective Repeat

The receiver buffers out-of-order frames inside its window and acknowledges each frame individually. The sender resends only the frame that was lost. Go-Back-N differs in the two lines marked below.

```
on frame arrival at receiver (sequence s):
    if s in receive window:
        mark s received; send ACK(s)
        if SELECTIVE_REPEAT: buffer s   # GBN discards if s != expected
        deliver all in-order frames from the window base
on timeout of frame s at sender:
    if SELECTIVE_REPEAT: resend only s   # GBN resends s .. last sent
    restart timer
```

3.3.3 TCP Reno fast recovery

On three duplicate ACKs, Reno halves the window and continues in congestion avoidance (cwnd = ssthresh = cwnd / 2), whereas Tahoe sets cwnd to 1 and slow-starts again. On a timeout both set cwnd to 1.

3.3.4 Priority queuing and weighted fair queuing

Priority queuing always serves the voice queue first. Weighted fair queuing uses the self-clocked approximation: every arriving packet gets a finish tag = max(virtual time, last tag of its flow) + size / weight, and the scheduler sends the packet with the smallest tag. Voice receives a configurable share (25% in the experiments), so bulk traffic cannot starve and voice cannot starve it either.

3.4 Novelty statement

The novelty is the combination, not a new protocol: (1) eight labs in one zero-install page, (2) the baseline and the improvement run side by side on identical seeded input, (3) a test suite of known answers that makes the numbers trustworthy, and (4) a scripted experiment runner that turns the same code into mean ± standard deviation tables. To the best of the author’s knowledge no student-facing tool for this syllabus offers all four together; this is a statement about the project’s aim and has not been checked by a systematic survey of every teaching tool.

3.5 Assumptions and design decisions

- **Simplified models.** TCP window behaviour is simulated per round-trip time. The shared-link model is a fluid model with 1 ms steps. Queuing is packet by packet on a single output port. These models show the mechanism, not the exact numbers of a real network.
- **Fixed seeds.** Each random run uses a mulberry32 generator, so a seed always reproduces the same result.

- **Plain web technology.** HTML, CSS and JavaScript only. No server and no internet are needed; fonts fall back to system fonts when offline.
- **Logic separated from display.** Every lab exports its logic so that tests can call it without a browser.
- **Infinity is 32.** RIP uses 16, but a legitimate cost of 20 in the example network would be clipped; the choice is documented in the lab.

4 Implementation

4.1 Tools, platform and versions

Table 4.1: Tools and versions

Item	Detail
Languages	HTML5, CSS3, JavaScript (ES2020), no framework, no build step
Run-time (user)	Any modern browser; open f1_team34_index.html
Test and experiment run-time	Node.js v22.23.4
Browser used for screenshots	Chromium 153 (headless, Playwright)
Machine used for the runs	Linux cloud container, 2 CPU cores, 7 GB memory; all 30-seed experiments finish in about 0.5 s
Version control	Git; repository https://github.com/savir18/CN
Typefaces	IBM Plex Sans and IBM Plex Mono (Google Fonts, system fallback offline)

4.2 Simulation setup and parameters

Table 4.2 lists the default parameters of the four comparisons. These are the values used in Chapter 5; the labs let the user change all of them.

Table 4.2: Parameters of the four experiments

Experiment	Parameters
Routing (count to infinity)	Chains of 3, 4, 5, 6 and 8 routers; every link cost 1; the last link is broken after the tables have converged; infinity = 32; synchronous rounds; deterministic, so one run per cell
Flow control (GBN against SR)	200 frames; window 8; frame time 1; one-way propagation 2 frame times; data-frame loss 0.02, 0.10, 0.20; ACKs not lost; automatic timeout = 2·Tp + 0.1; 30 seeds
TCP (Tahoe against Reno)	100 round-trip times; initial ssthresh 16; per-round loss probability 0.02, 0.05, 0.10 (80% of events are triple duplicate ACKs, 20% timeouts); identical loss events for both variants; 30 seeds
Queuing (FIFO, PQ, WFQ)	1 Mbps link; 20 s; buffer 50 packets per queue; voice 200 B every 20 ms; web Poisson 500 B (30% of data load); bulk on/off 1500 B (70% of data load, mean burst 0.3 s); data load 50%, 80%, 95%; WFQ voice weight 25%; identical traffic for all three; 30 seeds

4.3 Code structure

Table 4.3 shows the files. About 5,900 lines of JavaScript, CSS and tests make up the project, and every file name starts with `f1_team34_`.

Table 4.3: Project files

File	Role	Lines
f1_team34_index.html	Page shell; loads styles and scripts	–
css/f1_team34_styles.css	Design tokens, layout, light and dark themes	681
js/f1_team34_core.js	Namespace, seeded RNG, tab registry, router, theme, DOM and widget helpers	350
js/f1_team34_charts.js	Canvas line, bar and scatter charts	409
js/f1_team34_overview.js	Landing page and lab cards	168
js/f1_team34_journey.js	Encapsulation, TTL, checksums, switching delay, multiplexing	292
js/f1_team34_error_detection.js	Hamming, CRC, checksum, Monte Carlo test	833
js/f1_team34_flow_control.js	Stop-and-Wait, Go-Back-N, Selective Repeat simulator	653
js/f1_team34_multiple_access.js	ALOHA, CSMA and CSMA/CD models	486
js/f1_team34_subnetting.js	IPv4 arithmetic, VLSM, summarisation	314
js/f1_team34_routing.js	Dijkstra and distance vector	333
js/f1_team34_tcp_congestion.js	Tahoe, Reno, bottleneck model with drop-tail and RED	378
js/f1_team34_queuing_qos.js	FIFO, priority queuing, WFQ; VoIP verdict	295
tests/f1_team34_tests*.js	78 automated tests	–
scripts/f1_team34_experiments.js	30-seed experiment runner (new for this report)	–

4.4 Testing and verification

The test runner is home-made and has no dependency. `node tests/f1_team34_tests.js` runs 78 tests and prints “78 passed, 0 failed”. Table 4.4 splits them by lab, and Appendix B gives the complete output. Four tests check a known result that the code must reproduce: the CRC-32 of the text 123456789 is 0xCBF43926, the IPv4 header checksum example gives 0xB861, the peak throughput of pure ALOHA is 0.184 at G = 0.5, and Dijkstra and distance vector agree on the cost between every pair of routers. The other tests check invariants, for example that every frame arrives exactly once and in order under random loss, and that a seed repeats.

Table 4.4: Automated tests by lab

Lab	Tests	Examples of what is checked
Packet journey	8	CRC-32 check value; checksum 0xB861; TTL falls by 1 per router; 58 B header overhead; switching delay formulas; overload probability
Error detection	12	Hamming locates every single-bit error; textbook CRC remainder 1110; checksum weakness; Monte Carlo detection
Flow control	11	Ideal efficiency 1/(1+2a); one lost frame makes SR resend 1 and GBN resend the window; exactly-once, in-order delivery under loss
Multiple access	9	ALOHA peak 0.184 and slotted ALOHA peak 0.368; Kleinrock–Tobagi CSMA formulas; CSMA/CD recovers faster
Subnetting	11	Network, broadcast and host counts; VLSM 100/50/20/2 in a /24; summarisation with gaps

Lab	Tests	Examples of what is checked
Routing	6	Textbook Dijkstra distances; DV equals Dijkstra; count to infinity; split horizon and poison reverse
TCP congestion	12	Slow start and congestion avoidance; Tahoe and Reno after loss; Reno never sends fewer segments; fairness; RED against drop-tail
Queuing and QoS	9	FIFO against priority and WFQ; WFQ never starves bulk; delay grows with load under FIFO; repeatability

4.5 Challenges and how they were solved

Table 4.5: Problems met during the build and their fixes

Problem	Fix
Infinity of 16 clipped a legitimate path cost of 20 in the routing example.	Infinity raised to 32 and documented in the lab.
The packet-delay formula was wrong when the last packet was smaller than the others.	A single packet takes N times its own transmission time; otherwise (P + N − 2) full packets plus the last one, plus propagation.
The routing lab crashed on empty select lists for the destination view.	Select options are now built from the graph nodes; the default destination is the last router.
The first draft of the report claimed trends that no test backed up (Monte Carlo and CSMA).	Claims were rewritten to match what the tests actually show.
The numbers on the lab screens (default settings) differed from the settings used in earlier written results.	Chapter 5 states every parameter, and the runner script reproduces the tables.

5 Experimental Setup and Performance Evaluation

5.1 Experimental setup

All experiments run with `node scripts/f1_team34_experiments.js`, which uses the same logic modules as the browser labs and writes `results/f1_team34_experiment_results.json`. Each random cell is repeated with 30 seeds (seed = base × 1000 + i, for i = 1 to 30). The baseline and the improved mechanism receive identical conditions in each run: the same loss events, the same traffic, the same topology. The tables report the mean ± the sample standard deviation (n − 1) of the 30 runs. The routing experiment is deterministic, so a single run per cell is reported.

5.2 Scenarios and parameters

Each random experiment has three scenarios of increasing stress: S1 low, S2 medium and S3 high. The exact values are in Table 4.2. The metrics are chosen to match the purpose of each mechanism: rounds until stable for routing; link efficiency and retransmissions per frame for flow control; segments sent for TCP; 95th percentile delay and packet loss of the voice flow for queuing.

5.3 Results

5.3.1 Routing: split horizon against plain distance vector

Table 5.1: Rounds until the tables are stable after the last link of a chain breaks

Routers in the chain	Plain DV (baseline)	Split horizon	Poison reverse
3	31	2	2
4	31	3	3
5	31	4	4
6	31	5	5
8	31	7	7

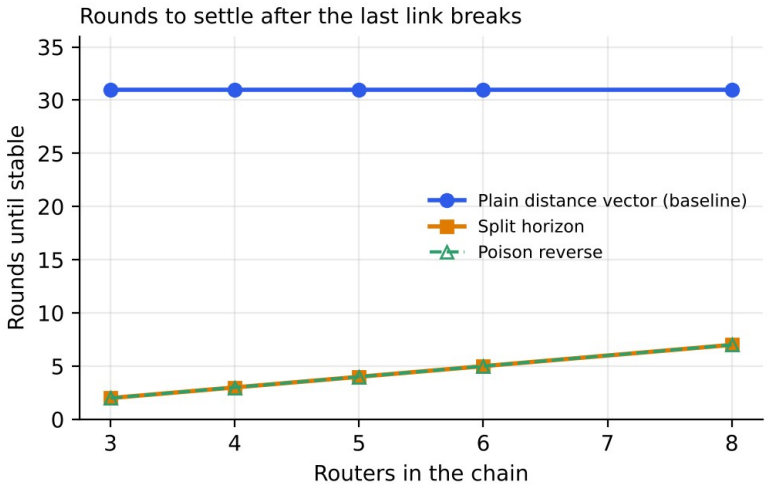


Figure 5.1: Rounds to settle against chain length

Plain distance vector needed 31 rounds in every chain, which is the point where the cost of the lost route reaches infinity (32). Split horizon and poison reverse needed n − 1 rounds, because the news that the last router is unreachable travels one hop per round.

5.3.2 Flow control: Selective Repeat against Go-Back-N

Table 5.2: Flow control, mean ± SD over 30 seeds (baseline: Go-Back-N)

Scenario	Metric	Baseline: Go-Back-N	Proposed: Selective Repeat	Improvement
S1 low loss	Link efficiency	0.869 ± 0.035	0.925 ± 0.019	+6.4%
	Retransmissions per frame	0.131 ± 0.045	0.023 ± 0.008	−82.3%
S2 medium loss	Link efficiency	0.610 ± 0.060	0.794 ± 0.032	+30.0%
	Retransmissions per frame	0.633 ± 0.162	0.111 ± 0.024	−82.4%
S3 high loss	Link efficiency	0.405 ± 0.047	0.681 ± 0.033	+68.0%
	Retransmissions per frame	1.464 ± 0.272	0.245 ± 0.032	−83.3%

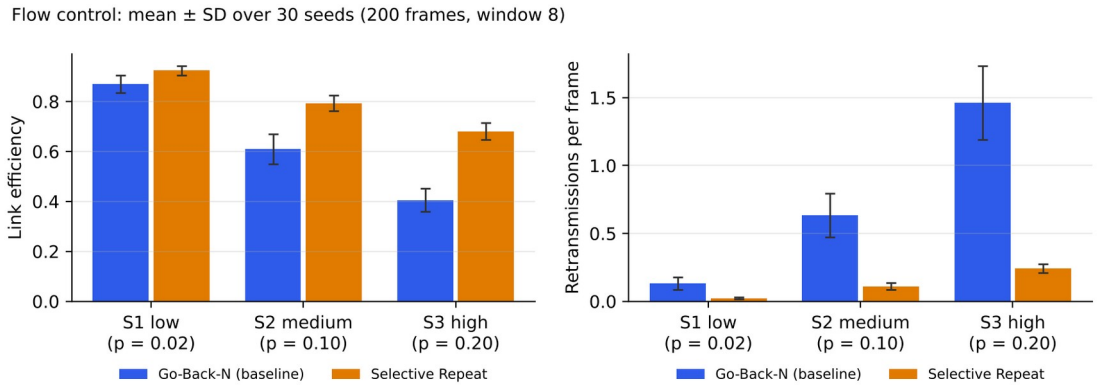


Figure 5.2: Efficiency and retransmissions, Go-Back-N against Selective Repeat

5.3.3 TCP: Reno against Tahoe

Table 5.3: TCP, mean ± SD over 30 seeds (baseline: Tahoe)

Scenario	Metric	Baseline: Tahoe	Proposed: Reno	Improvement
S1 low loss	Segments sent in 100 RTTs	3987 ± 1366	4322 ± 1199	+8.4%
S2 medium loss	Segments sent in 100 RTTs	2498 ± 1140	2924 ± 1044	+17.0%
S3 high loss	Segments sent in 100 RTTs	1276 ± 518	1620 ± 545	+27.0%

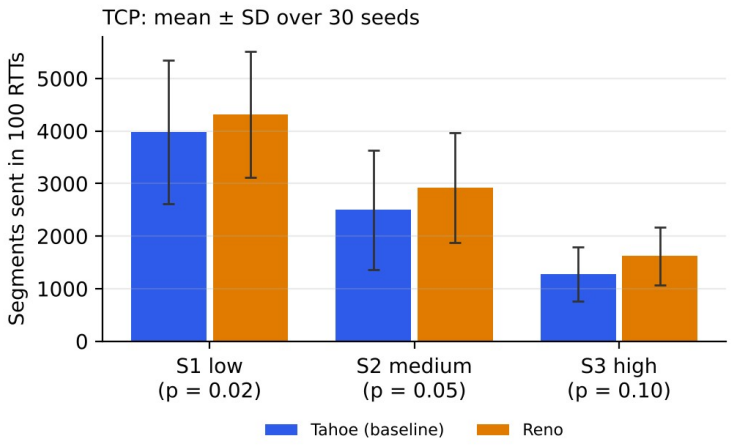


Figure 5.3: Segments sent in 100 round-trip times, Tahoe against Reno

5.3.4 Queuing: weighted fair queuing against FIFO

Table 5.4: Voice flow on a 1 Mbps link, mean ± SD over 30 seeds (baseline: FIFO)

Scenario	Metric	Baseline: FIFO	Proposed: WFQ	Improvement
S1 light data load	Voice p95 delay (ms)	16.6 ± 1.6	12.6 ± 0.3	−24.0%
	Voice loss (%)	0.00 ± 0.00	0.00 ± 0.00	no loss either way
S2 medium data load	Voice p95 delay (ms)	338.1 ± 24.3	15.8 ± 0.3	−95.3%
	Voice loss (%)	2.60 ± 1.39	0.00 ± 0.00	−100.0%
S3 heavy data load	Voice p95 delay (ms)	374.3 ± 10.5	16.2 ± 0.3	−95.7%
	Voice loss (%)	8.57 ± 2.33	0.00 ± 0.00	−100.0%

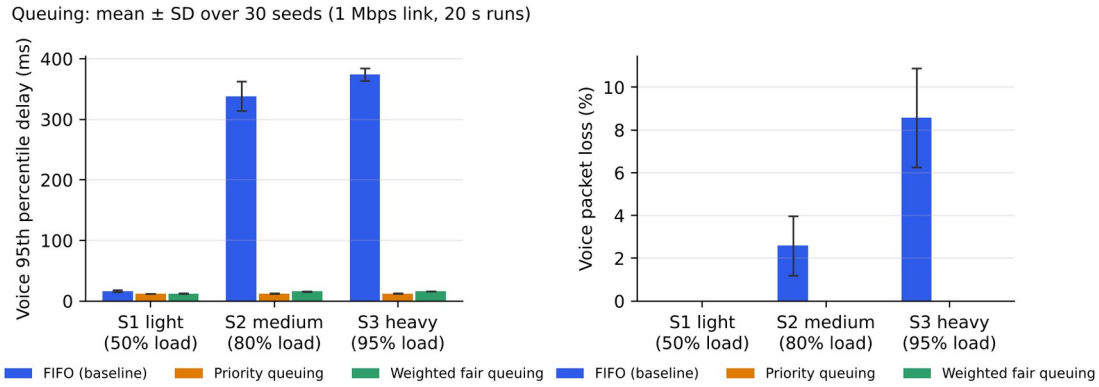


Figure 5.4: Voice delay and loss under FIFO, priority queuing and WFQ

Table 5.5: Priority queuing against WFQ, and the effect on bulk traffic (mean ± SD)

Scenario	Voice p95 (ms): PQ	Voice p95 (ms): WFQ	Bulk loss % PQ	Bulk loss % WFQ	Bulk loss % FIFO
S1 light data load	12.0 ± 0.3	12.6 ± 0.3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
S2 medium data load	12.6 ± 0.2	15.8 ± 0.3	2.07 ± 2.04	2.00 ± 2.01	4.31 ± 2.13

Scenario	Voice p95 (ms): PQ	Voice p95 (ms): WFQ	Bulk loss % PQ	Bulk loss % WFQ	Bulk loss % FIFO
S3 heavy data load	12.6 ± 0.2	16.2 ± 0.3	14.68 ± 5.10	13.46 ± 4.95	14.57 ± 3.21

5.4 Analysis

- **Routing.** Split horizon fixes the two-router loop on a chain. It settled faster than plain DV in every chain tested (2 to 7 rounds against 31).
- **Flow control.** Go-Back-N wastes bandwidth in proportion to the window it repeats. Selective Repeat's retransmissions stay near the loss probability (0.023 at 2% loss, 0.111 at 10%, 0.245 at 20%), while Go-Back-N's grow to 1.464 retransmissions per frame at 20% loss. The efficiency gain grows with loss, from +6.4% to +68.0%. The standard deviations do not overlap in S2 and S3.
- **TCP.** Reno sent 8.4% more segments at low loss and 27.0% more at high loss. The standard deviations are large (for example 3987 ± 1366 for Tahoe in S1), because one early loss changes the whole window history. The advantage is consistent in direction but the per-seed variation is wide; Section 5.5 explains why the size of the gain must be read with care.
- **Queuing.** Under light load all three rules keep voice delay below 17 ms. At 80% and 95% data load FIFO's p95 voice delay is 338.1 and 374.3 ms, well over the 150 ms limit, with 2.60% and 8.57% loss. Priority queuing and WFQ hold the delay at 12 to 16 ms and remove voice loss.
- **Priority queuing against WFQ.** Priority has the lowest voice delay (12.6 ms against 15.8 and 16.2 ms). WFQ pays about 3 to 4 ms for a guaranteed share of the link for the other flows, which is why it is preferred when starvation must be avoided. Bulk loss under both rules is no higher than under FIFO within one standard deviation (2.1% and 2.0% against 4.3% at 80% load; 14.7% and 13.5% against 14.6% at 95%), so protecting voice did not cost the data flows extra loss in these runs.

5.5 Critical evaluation

- **Model limits.** The TCP window model draws a loss event per round, independent of the window size. In a real network a larger window sees more losses, so the Reno advantage here is a property of the model; the absolute percentages should not be quoted as real-world numbers. The shared-link lab in the project uses a different, queue-based model, and was not used for these tables.
- **Split horizon is not a general cure.** The chain topology has only two-router loops. Split horizon and poison reverse do not stop larger loops in meshed networks, and the experiment does not test those.
- **Flow control details.** ACKs are not lost and the timeout is fixed at $2 \cdot Tp + 0.1$, so the experiment isolates data-frame loss. Results with ACK loss, or a tighter timeout, would differ.
- **Queuing.** A single voice call with a fixed packet rate and one output port. Priority queuing could starve other traffic if voice load were high; this was not tested. Jitter is reported by the lab but not in Chapter 5, which concentrates on delay and loss.
- **Statistics.** 30 seeds give a stable mean, but no significance tests were run; the claim is that the means differ by many standard deviations in S2 and S3 of flow control and queuing, not that every gap is statistically proven.

- **No user study.** The report does not claim that the tool improves learning; it was not tested with students.

6 Conclusion and Future Work

6.1 Summary of results

Protocol Lab turned the syllabus into eight working labs in a single page of plain web code. Four baseline-versus-improvement comparisons were measured over 30 seeds. Split horizon removes the count-to-infinity delay (2 to 7 rounds against 31). At 20% loss Selective Repeat improves link efficiency from 0.405 ± 0.047 to 0.681 ± 0.033 and cuts retransmissions per frame from 1.464 to 0.245. At 95% data load WFQ reduces the 95th percentile voice delay from 374.3 ± 10.5 ms to 16.2 ± 0.3 ms and removes the 8.57% voice loss. Reno sends 8% to 27% more segments than Tahoe in the model. All 78 tests pass, including four known-answer checks.

6.2 Limitations

- Simplified models (per-RTT TCP, fluid shared link, single-port queue); IEEE 802.11 CSMA/CA is theory only; OSPF, RIP and BGP appear as algorithms, not as protocol messages.
- Unit 7 (DNS, HTTP, FTP) has no lab.
- No tests on real devices beyond headless Chromium; no user study; no significance testing.

6.3 Future work

- A DNS lab with recursive and iterative lookups and caching, to cover Unit 7.
- A TCP handshake and UDP comparison; CSMA/CA for IEEE 802.11.
- Larger routing topologies where split horizon fails, and triggered updates.
- Export of lab results to CSV and a short classroom trial with students to measure learning.

7 References

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[2] A. S. Tanenbaum, N. Feamster and D. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.

[3] L. L. Peterson and B. S. Davie, *Computer Networks: A Systems Approach*, 6th ed. Morgan Kaufmann, 2021.

[4] T. R. Henderson, M. Lacage and G. F. Riley, "Network simulations with the ns-3 simulator," SIGCOMM Demonstration, 2008.

[5] B. Lantz, B. Heller and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010.

[6] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.

[7] W. Stevens, "TCP slow start, congestion avoidance, fast retransmit, and fast recovery algorithms," RFC 2001, IETF, 1997.

[8] K. Fall and S. Floyd, "Simulation-based comparisons of Tahoe, Reno and SACK TCP," *ACM SIGCOMM Computer Communication Review*, vol. 26, no. 3, pp. 5–21, 1996.

[9] A. Demers, S. Keshav and S. Shenker, "Analysis and simulation of a fair queueing algorithm," in *Proc. ACM SIGCOMM*, 1989.

[10] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993.

[11] C. Hedrick, "Routing Information Protocol," RFC 1058, IETF, 1988.

[12] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[13] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proceedings of the IRE*, vol. 49, no. 1, pp. 228–235, 1961.

[14] N. Abramson, "The ALOHA system: another alternative for computer communications," in *Proc. AFIPS Fall Joint Computer Conference*, 1970.

[15] ITU-T, "One-way transmission time," Recommendation G.114, 2003.

Appendix A: Code and configuration listings

Listings are copied from the repository at build time, so they match the code that produced the results.

A.1 Dijkstra with a step trace (js/f1_team34_routing.js)

```
R.dijkstra = function (g, src) {
  const ids = g.nodes.map((n) => n.id);
  const dist = {}; const prev = {}; const done = {};
  ids.forEach((id) => { dist[id] = Infinity; prev[id] = null; done[id] = false; });
  dist[src] = 0;
  const snap = () => ({ dist: Object.assign({}, dist), prev: Object.assign({}, prev), done:
Object.assign({}, done) });
  const steps = [Object.assign({ picked: null, relax: [] }, snap())];
  for (;;) {
    let u = null;
    ids.forEach((id) => { if (!done[id] && dist[id] < Infinity && (u === null || dist[id] <
dist[u])) u = id; });
    if (u === null) break;
    done[u] = true;
    const relax = [];
    R.neighbours(g, u).forEach((nb) => {
      if (done[nb.id]) return;
      const alt = dist[u] + nb.cost;
      const better = alt < dist[nb.id];
      relax.push({ node: nb.id, alt, old: dist[nb.id], improved: better });
      if (better) { dist[nb.id] = alt; prev[nb.id] = u; }
    });
    steps.push(Object.assign({ picked: u, relax }, snap()));
  }
  return { dist, prev, steps };
```

A.2 One distance vector round with split horizon and poison reverse

```
R.dvStep = function (g, st, mode) {
  const ids = g.nodes.map((n) => n.id); const next = {}; let changed = false;
  ids.forEach((n) => {
    next[n] = {}; const nbs = R.neighbours(g, n);
    ids.forEach((d) => {
      if (d === n) { next[n][d] = { cost: 0, next: n }; return; }
      let best = { cost: INF, next: null };
      nbs.forEach((nb) => {
        let adv = st[nb.id][d].cost;
        if (st[nb.id][d].next === n && d !== nb.id) { // the neighbour reaches d through us
          if (mode === 'split') adv = INF; // do not tell us the route we taught it
          if (mode === 'poison') adv = INF; // tell us explicitly it is unreachable
        }
        const c = Math.min(INF, nb.cost + adv);
        const cur = st[n][d];
        if (c < best.cost || (c === best.cost && c < INF && (nb.id === cur.next && best.next
!== cur.next || (best.next !== cur.next && nb.id < best.next)))) best = { cost: c, next:
nb.id };
      });
      if (best.cost >= INF) best = { cost: INF, next: null };
      next[n][d] = best;
      if (best.cost !== st[n][d].cost || best.next !== st[n][d].next) changed = true;
```

```
    });
  });
  return { state: next, changed };
```

A.3 TCP congestion window per round (js/f1_team34_tcp_congestion.js)

```
TCP.windowTrace = function (cfg) {
  const o = Object.assign({ variant: 'reno', rounds: 24, ssthresh: 16, events: {}, maxCwnd:
1000 }, cfg);
  let cwnd = 1; let ssthresh = o.ssthresh;
  const rows = [];
  for (let r = 0; r <= o.rounds; r++) {
    const phase = cwnd < ssthresh ? 'slow start' : 'congestion avoidance';
    const ev = o.events[r] || null;
    rows.push({ round: r, cwnd, ssthresh, phase, event: ev });
    if (r === o.rounds) break;
    if (ev === 'timeout') { ssthresh = Math.max(Math.floor(cwnd / 2), 2); cwnd = 1; }
    else if (ev === 'dup') {
      ssthresh = Math.max(Math.floor(cwnd / 2), 2);
      cwnd = o.variant === 'tahoe' ? 1 : ssthresh; // Reno: fast recovery, carry on linearly
from half
    } else if (cwnd < ssthresh) cwnd = Math.min(cwnd * 2, ssthresh); // slow start doubles
every RTT
    else cwnd += 1; // congestion avoidance adds one segment per RTT
    cwnd = Math.min(cwnd, o.maxCwnd);
  }
  const segments = rows.reduce((s, x) => s + x.cwnd, 0);
  return { rows, segments };
```

A.4 Running all three queue rules on identical traffic (js/f1_team34_queuing_qos.js)

```
Q.compare = function (cfg) {
  const arr = Q.generate(cfg);
  const out = {};
  Object.keys(Q.DISCIPLINES).forEach((d) => { out[d] = Q.run(arr, Object.assign({}, cfg,
{ discipline: d })); });
  return out;
```

A.5 Experiment runner, flow control part (scripts/f1_team34_experiments.js)

```
/* E2  Flow control: Go-Back-N (baseline) vs Selective Repeat, random data-frame loss */
out.flow = [];
for (const [name, p] of [['S1 low loss', 0.02], ['S2 medium loss', 0.10], ['S3 high loss',
0.20]]) {
const cell = { scenario: name, loss: p };
for (const proto of ['gbn', 'sr']) {
  const effs = []; const retx = [];
  seeds(1).forEach((seed) => {
    const r = CN.flowControl.simulate({ protocol: proto, N: 200, W: 8, Tf: 1, Tp: 2, pData:
p, pAck: 0, seed, record: false });
    if (r.finish !== null) { effs.push(r.stats.efficiency); retx.push(r.stats.retx /
r.stats.N); }
  });
  cell[proto] = { efficiency: stats(effs), retxPerFrame: stats(retx) };
}
```

```
out.flow.push(cell);
```

A.6 How to reproduce every result

```
git clone https://github.com/savir18/CN.git
cd CN
node tests/f1_team34_tests.js              # 78 passed, 0 failed
node scripts/f1_team34_experiments.js      # Chapter 5 tables, about 0.5 s
# open f1_team34_index.html in a browser for the labs
```

Appendix B: Screenshots, raw results and test output

B.1 Screenshots of the eight labs

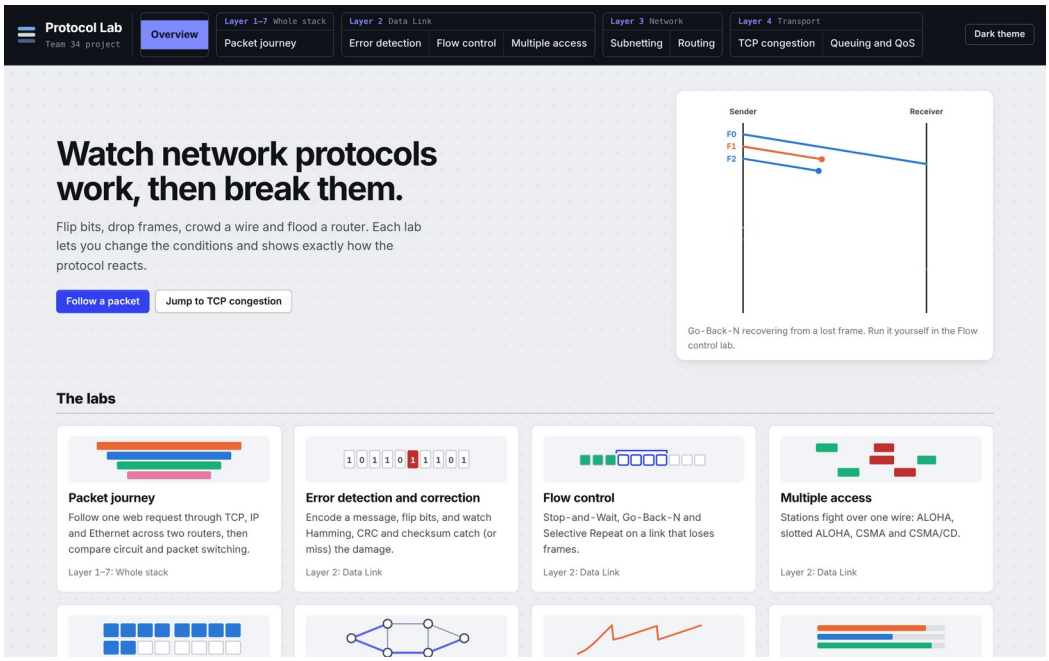


Figure B.1: Landing page (light theme, 1440 px wide)

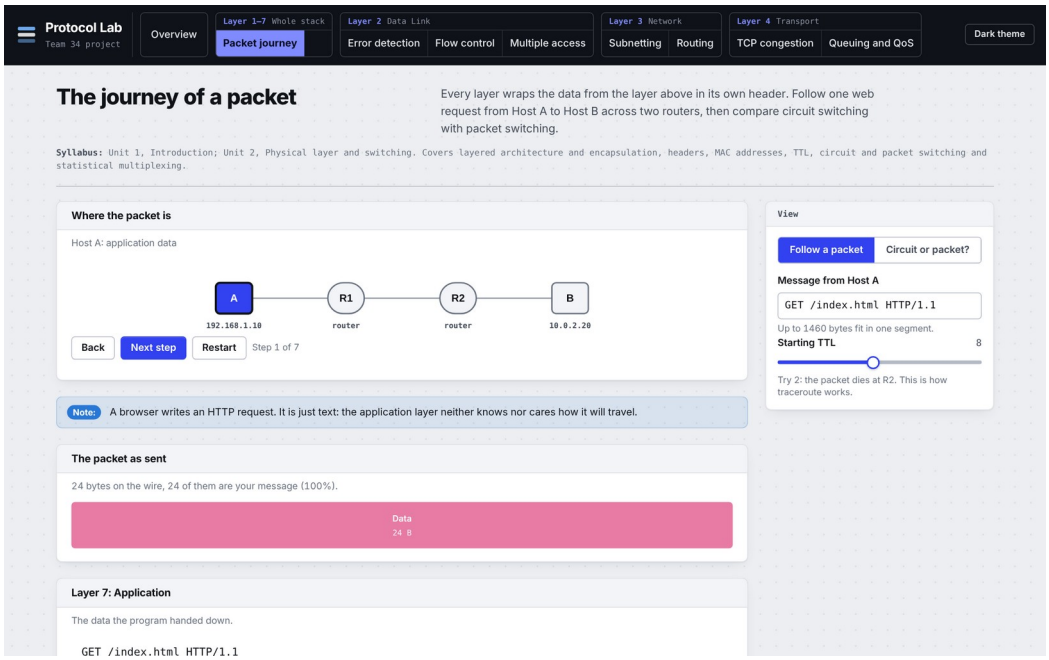


Figure B.2: Packet journey (light theme, 1440 px wide)

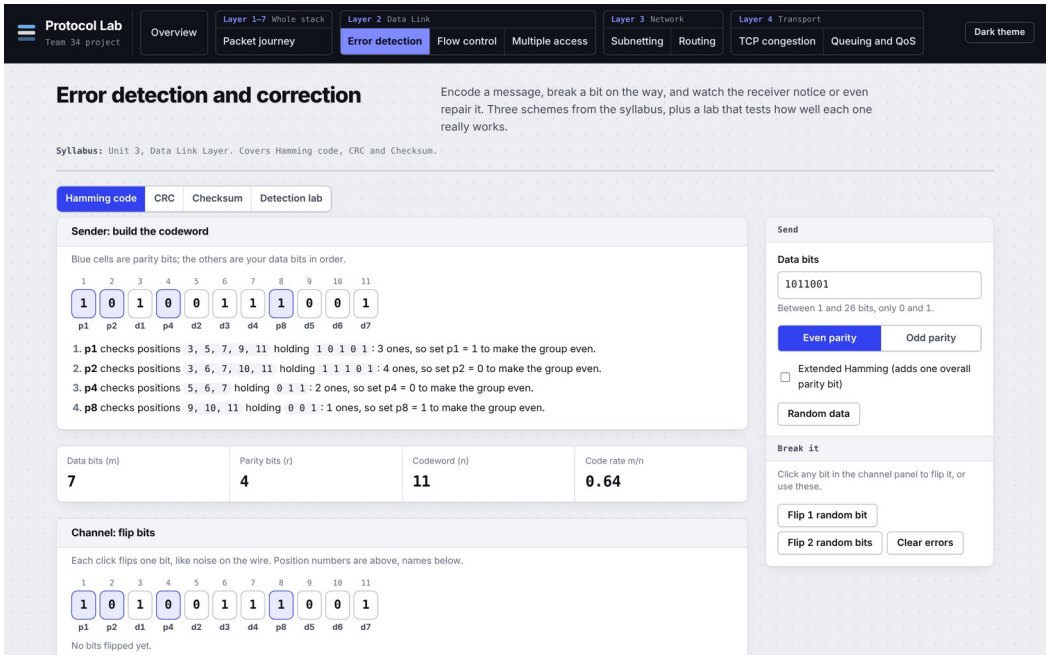


Figure B.3: Error detection (light theme, 1440 px wide)

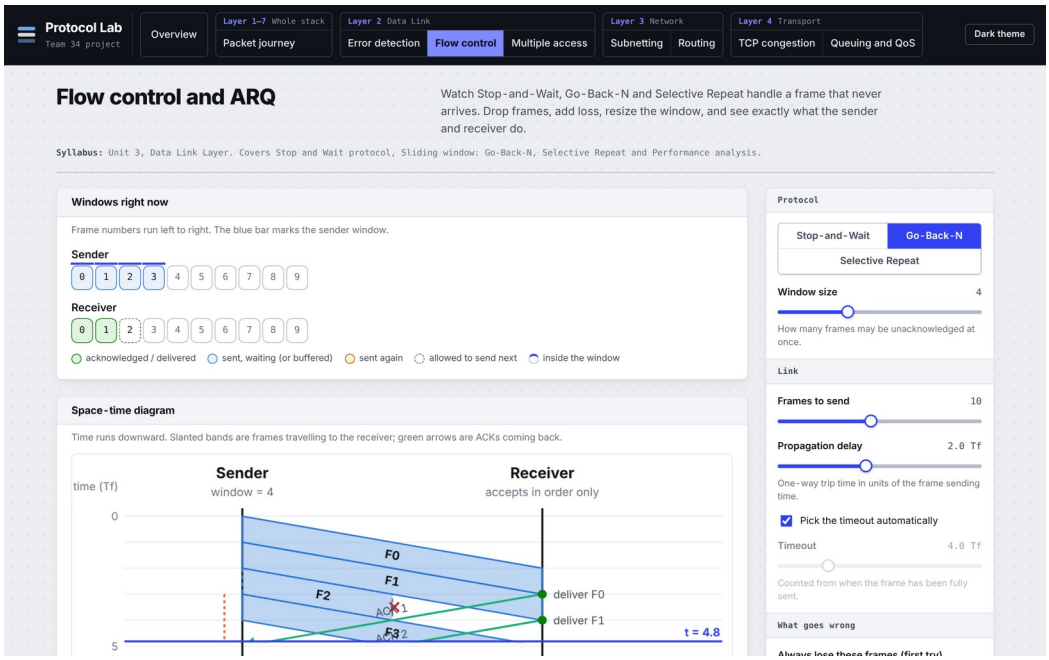


Figure B.4: Flow control (light theme, 1440 px wide)

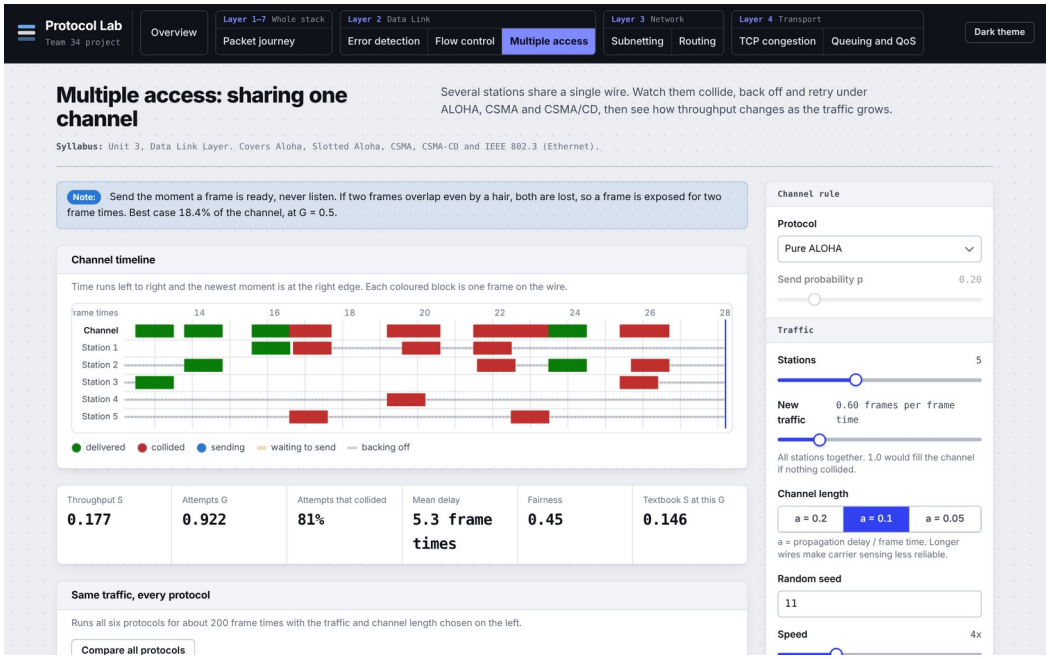


Figure B.5: Multiple access (light theme, 1440 px wide)

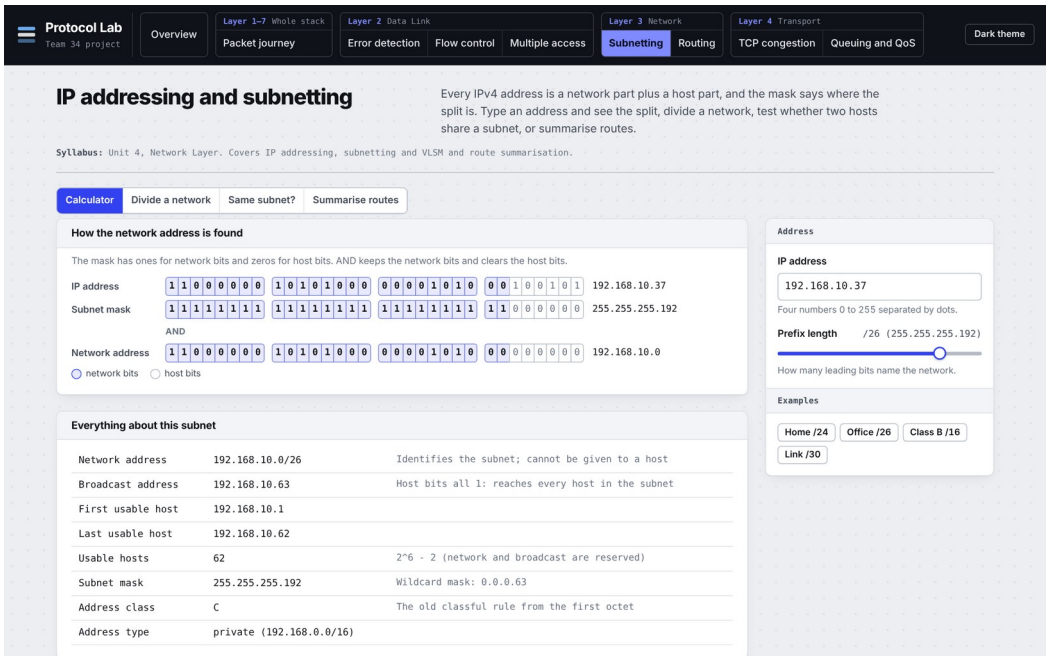


Figure B.6: Subnetting (light theme, 1440 px wide)

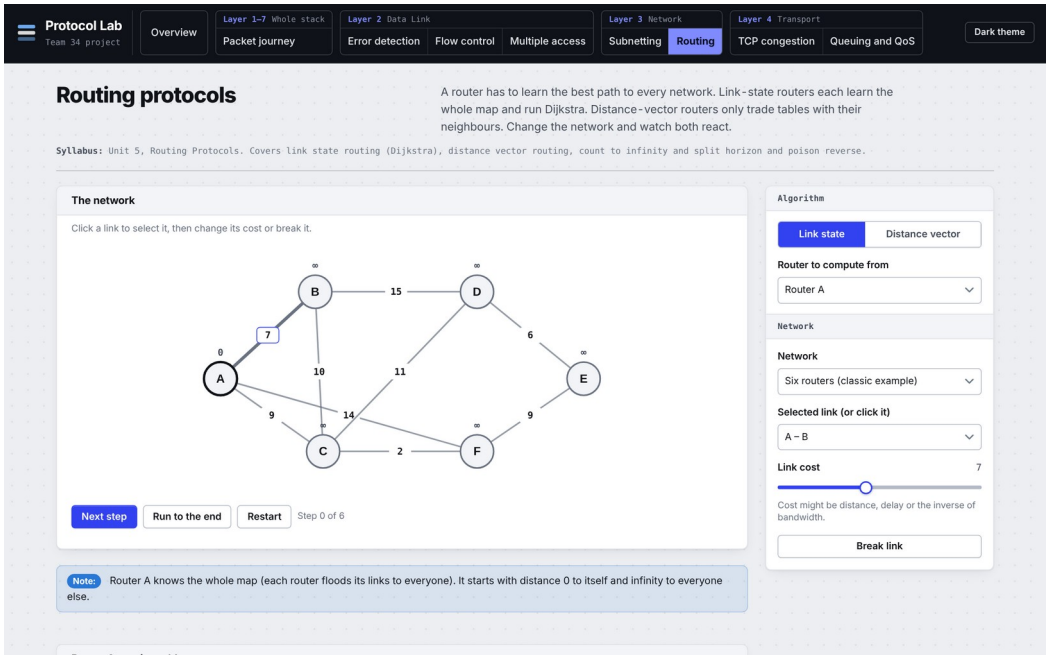


Figure B.7: Routing (light theme, 1440 px wide)

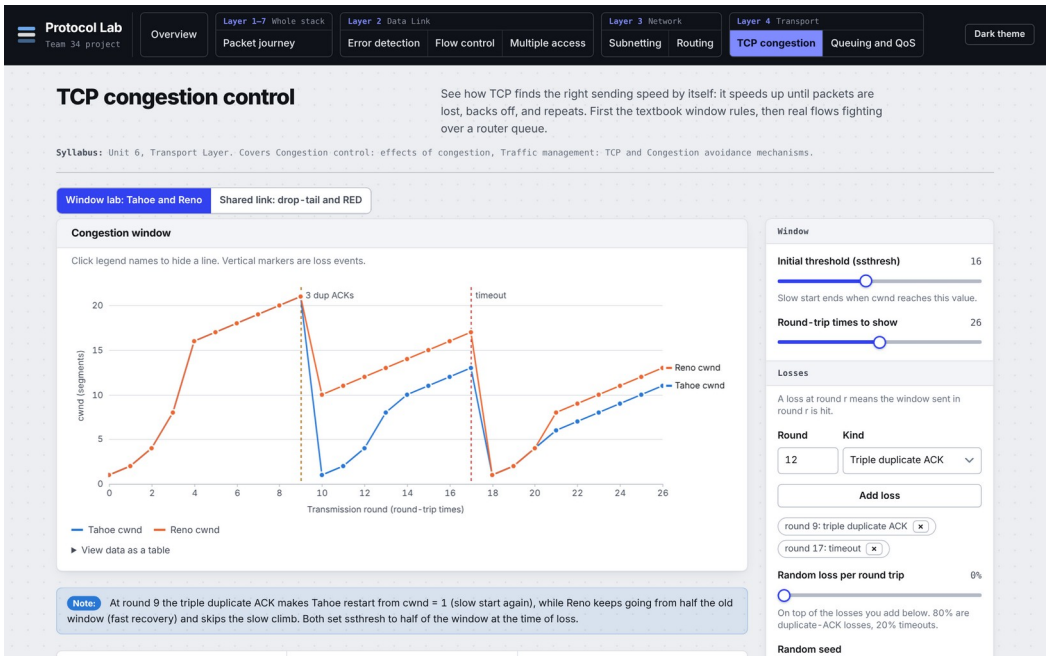


Figure B.8: TCP congestion (light theme, 1440 px wide)

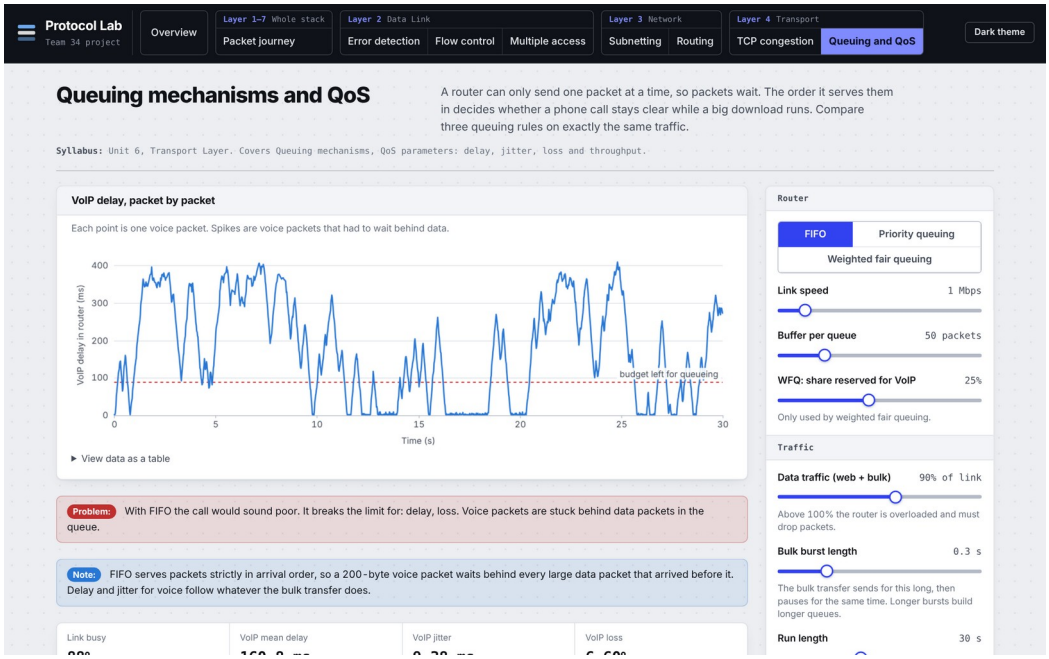


Figure B.9: Queuing and QoS (light theme, 1440 px wide)

B.2 Test output

```
Error detection
  ok   Hamming: parity bit count follows 2^r >= m + r + 1
  ok   Hamming: 7 data bits need 4 parity bits and decode cleanly
  ok   Hamming: every single-bit error is located and corrected (even and odd parity)
  ok   Hamming: plain code mis-corrects a double error
  ok   Extended Hamming: single errors corrected, double errors detected, overall bit error
found
  ok   CRC: textbook example 1101011011 / x^4+x+1 gives remainder 1110
  ok   CRC: encode then check gives zero remainder; shift-register form matches long
division
  ok   CRC: every single-bit error is detected; a multiple of G(x) is not
  ok   CRC: guarantees table is truthful (bursts <= r always caught)
  ok   Checksum: ones complement sum with end-around carry, verify passes
  ok   Checksum: reordering words is not detected (known weakness)
  ok   Monte Carlo: parity never catches even-sized errors; CRC-16 catches every burst up to
16

flow control
  ok   Stop-and-Wait with no loss: one frame per round trip, efficiency 1/(1+2a)
  ok   Go-Back-N without loss: efficiency approaches min(1, W/(1+2a))
  ok   Selective Repeat without loss behaves like Go-Back-N
  ok   One lost frame: Selective Repeat resends exactly 1, Go-Back-N resends the window
  ok   Lost ACK: Stop-and-Wait resends the frame, receiver recognises the duplicate
  ok   Cumulative ACKs: a lost ACK in Go-Back-N is covered by the next one
  ok   Every frame is delivered exactly once and in order, for all protocols under random
loss
  ok   Same seed gives the same result; scripted drops use only the first attempt
  ok   Simulation cannot loop forever when everything is lost
  ok   Measured efficiency under loss matches the textbook approximations (large window)
  ok   parseList accepts numbers and rejects junk

multiple access
  ok   Pure ALOHA simulation follows S = G e^(-2G); maximum about 0.184 at G = 0.5
```

```
  ok   Slotted ALOHA simulation follows S = G e^(-G); maximum about 0.368 at G = 1
  ok   Non-persistent and 1-persistent CSMA simulations match the Kleinrock-Tobagi slotted
formulas
  ok   Carrier sensing beats ALOHA at the same load; shorter wires (smaller a) help CSMA
  ok   CSMA/CD recovers from collisions faster than plain 1-persistent CSMA at heavy load
  ok   Live simulation: light traffic is delivered almost completely; every protocol
conserves frames
  ok   Live simulation: heavy traffic makes ALOHA collide a lot, CSMA/CD copes better
  ok   Live simulation: slotted ALOHA frames always start on a slot boundary
  ok   Live simulation is repeatable for a given seed

tcp congestion
  ok   slow start doubles each round until ssthresh, then adds one
  ok   phase label switches at ssthresh
  ok   triple duplicate ACK: Tahoe restarts at 1, Reno halves
  ok   timeout sends both variants to 1
  ok   Reno never sends fewer segments than Tahoe on the same losses
  ok   makeEvents is deterministic and keeps scripted events
  ok   one Reno flow with a BDP-sized buffer keeps the link busy
  ok   a tiny buffer lowers utilisation
  ok   two equal flows share fairly
  ok   a longer RTT gets a smaller share
  ok   RED gives lower mean queueing delay than drop-tail
  ok   identical seeds give identical results

queuing qos
  ok   traffic generator produces sorted arrivals from all three flows near the requested
load
  ok   a lone packet sees only its own transmission time
  ok   every packet is either delivered or dropped, in all three rules
  ok   FIFO: voice shares the data queue, priority and WFQ protect it
  ok   priority keeps voice delay to roughly one packet time under overload
  ok   WFQ never starves the bulk flow
  ok   delay for voice grows with data load under FIFO
  ok   throughput never exceeds link speed and results are repeatable
  ok   verdict flags poor voice quality

subnetting
  ok   parse and print addresses; reject bad ones
  ok   masks from prefixes and back; non-contiguous masks rejected
  ok   classic worked example: 192.168.10.37/26
  ok   edge prefixes /30 /31 /32 and /0
  ok   detects network and broadcast addresses
  ok   same-subnet test
  ok   equal split rounds up to a power of two and tiles the space
  ok   VLSM: 100, 50, 20, 2 hosts in a /24
  ok   VLSM reports when the space is too small
  ok   route summarisation: four /24s become one /22; gaps are reported
  ok   parseCidr accepts slash and dotted-mask forms

routing
  ok   Dijkstra on the classic graph gives the textbook distances from A
  ok   Dijkstra trace finalises every reachable router once, in order of distance
  ok   a broken link is ignored and unreachable routers stay at infinity
  ok   distance vector converges to the same costs as Dijkstra
  ok   plain distance vector counts to infinity after the C link fails
  ok   split horizon and poison reverse stop the loop on the chain
```

```
journey
  ok   CRC-32 matches the standard check value
  ok   internet checksum matches a known IPv4 header (0xB861)
  ok   journey: TTL drops by one per router, MACs change per link, IPs stay
  ok   journey: TTL 2 dies at R2 and TTL 1 dies at R1
  ok   header overhead is 58 bytes; corrupted frame fails its FCS
  ok   packet switching: one packet and equal packets follow N*L/R and (N+P-1)*L/R
  ok   circuit delay is set-up + transmission + propagation; smaller last packet is not
slower than a full one
  ok   statistical multiplexing: 35 users at 10% on a link for 10 gives well under 0.1%
overload

78 passed, 0 failed
```

B.3 Raw results (excerpt of results/f1_team34_experiment_results.json)


```
{
  "runsPerCell": 30,
  "routing": [
    {
      "routers": 3,
      "none": 31,
      "split": 2,
      "poison": 2
    },
    {
      "routers": 4,
      "none": 31,
      "split": 3,
      "poison": 3
    },
    {
      "routers": 5,
      "none": 31,
      "split": 4,
      "poison": 4
    },
    {
      "routers": 6,
      "none": 31,
      "split": 5,
      "poison": 5
    },
    {
      "routers": 8,
      "none": 31,
      "split": 7,
      "poison": 7
    }
  ],
  "flow_S3": {
    "loss": 0.2,
    "gbn_efficiency": {
      "mean": 0.4053,
      "sd": 0.0468,
      "n": 30
    },
```

```
    "sr_efficiency": {
      "mean": 0.6808,
      "sd": 0.0334,
      "n": 30
    }
  },
  "qos_S3": {
    "dataLoad": 0.95,
    "fifo_p95": {
      "mean": 374.2910,
      "sd": 10.4778,
      "n": 30
    },
    "wfq_p95": {
      "mean": 16.2231,
      "sd": 0.2696,
      "n": 30
    }
  }
}
```